

Bash Scripting - Extended Deep Dive

Numeric vs String Comparisons in Bash

In Bash, the way you compare values depends on their type ? numeric or string.

This is because Bash treats everything as a string by default, but arithmetic operations are handled differently.

1. Numeric comparison operators (-eq, -ne, -lt, -gt, etc.)

These are used **inside [] or test** to compare numeric (integer) values.

Examples:

```
a=5
b=10

if [ $a -eq $b ]; then
    echo "Equal"
else
    echo "Not equal"
fi
```

Here `-eq` means 'equal numerically'. Similarly, you can use `-ne` (not equal), `-gt` (greater than), `-lt` (less than), etc.

2. String comparison operator (= or !=)

These are used to compare **text strings**, not numbers.

Example:

```
str1="hello"
str2="world"

if [ "$str1" = "$str2" ]; then
    echo "Same"
else
    echo "Different"
fi
```

If you mistakenly use `=` for numbers, Bash will treat them as **text**, not numbers, leading to incorrect comparisons.

3. Safer numeric comparison: (())

You can also use arithmetic context `(( ))` for numeric comparisons. It looks cleaner and doesn't require `-eq`.

Bash Scripting - Extended Deep Dive

```
a=5; b=10
if (( a < b )); then
    echo "a is less than b"
fi
```

Why not use '=' for numbers?

Because '=' compares **text**, not numeric values. For example:

```
a=10
b=2
[ "$a" = "$b" ] && echo Equal || echo NotEqual
```

This checks if the string '10' is exactly the same as '2' ? which is false ? but it doesn't understand which number is larger.

In contrast:

```
[ "$a" -gt "$b" ] && echo Greater
```

`-gt` correctly interprets 10 as greater than 2 numerically.

Summary: Choosing the Correct Comparison

Data Type	Recommended Syntax	Example
String	<code>["\$a" = "\$b"]</code> <code>if ["\$user" = "admin"]; then ...</code>	
Numeric	<code>[\$a -eq \$b]</code> or <code>((a == b))</code> <code>if [\$a -gt 10]; then ...</code>	

? Use `=` and `!=` for **strings**.

? Use `-eq`, `-ne`, `-gt`, `-lt`, etc. for **numbers**.

? Use `(( ))` for **clean numeric logic** (no need for ``` signs inside).

In summary, **Bash separates numeric and string operators** for safety and clarity, ensuring that each operation behaves as expected.